CS 224N Spring 2024 Assignment 4
Self-Attention, Transformers, and Pretraining

Note. Here are some things to keep in mind as you plan your time for this assignment.

- There are math questions again!
- The total amount of PyTorch code to write, and code complexity, of this assignment is lower than Assignment 3. However, you're also given less guidance or scaffolding in how to write the code.
- This assignment involves a pretraining step that takes approximately 1 hour to perform on GCP, and you'll have to do it twice. Colab set-up notebook has been provided similar to Assignment 4. The 1 hour timeline is an upper bound on the training time assuming older/slower GPU. On faster GPUs, the pretraining can finish in around 30-40 minutes.

This assignment is an investigation into Transformer self-attention building blocks, and the effects of pretraining. It covers mathematical properties of Transformers and self-attention through written questions. Further, you'll get experience with practical system-building through repurposing an existing codebase. The assignment is split into a written (mathematical) part and a coding part, with its own written questions. Here's a quick summary:

1. **Mathematical exploration:** What kinds of operations can self-attention easily implement? Why should we use fancier things like multi-headed self-attention? This section will use some mathematical investigations to illuminate a few of the motivations of self-attention and Transformer networks. **Note:** for all questions, you should justify your answer with mathematical reasoning when required.

2. **Extending a research codebase:** In this portion of the assignment, you'll get some experience and intuition for a cutting-edge research topic in NLP: teaching NLP models facts about the world through pretraining, and accessing that knowledge through finetuning. You'll train a Transformer model to attempt to answer simple questions of the form "Where was person [x] born?" – without providing any input text from which to draw the answer. You'll find that models are able to learn some facts about where people were born through pretraining, and access that information during fine-tuning to answer the questions.

   Then, you'll take a harder look at the system you built, and reason about the implications and concerns about relying on such implicit pretrained knowledge.

1. Attention Exploration (14 points)

Multi-head self-attention is the core modeling component of Transformers. In this question, we'll get some practice working with the self-attention equations, and motivate why multi-headed self-attention can be preferable to single-headed self-attention.

Recall that attention can be viewed as an operation on a *query* vector $q \in \mathbb{R}^d$, a set of *value* vectors $\{v_1, \ldots, v_n\}, v_i \in \mathbb{R}^d$, and a set of *key* vectors $\{k_1, \ldots, k_n\}, k_i \in \mathbb{R}^d$, specified as follows:

$$c = \sum_{i=1}^{n} v_i \alpha_i \tag{1}$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)} \tag{2}$$

with $alpha = \{\alpha_1, \ldots, \alpha_n\}$ termed the "attention weights". Observe that the output $c \in \mathbb{R}^d$ is an average over the value vectors weighted with respect to α.

(a) (3 points) **Copying in attention.** One advantage of attention is that it's particularly easy to "copy" a value vector to the output c. In this problem, we'll motivate why this is the case.

i. (2 points) The distribution α is typically relatively "diffuse"; the probability mass is spread out between many different α_i. However, this is not always the case. **Describe** (in one sentence) under what conditions the categorical distribution α puts almost all of its weight on some α_j, where $j \in \{1, \ldots, n\}$ (i.e. $\alpha_j \gg \sum_{i \neq j} \alpha_i$). What must be true about the query q and/or the keys $\{k_1, \ldots, k_n\}$?

Answer: When the score $k_j^\top q$ is much larger than every other score $k_i^\top q$ for $i \neq j$ — i.e. when the query q is strongly aligned with the key k_j and (nearly) orthogonal to or negatively aligned with all the other keys — the gap $k_j^\top q - \max_{i \neq j} k_i^\top q$ is large, and the exponentials inside the softmax make α_j dominate all the other weights, so $\alpha_j \to 1$.

ii. (1 point) Under the conditions you gave in (i), **describe** the output c.

Answer: Then $c \approx v_j$: since $\alpha_j \approx 1$ and $\alpha_i \approx 0$ for $i \neq j$, the output of attention is essentially the value vector v_j itself — attention "copies" v_j to the output.

(b) (2 points) **An average of two.** Instead of focusing on just one vector v_j, a Transformer model might want to incorporate information from *multiple* source vectors.

Consider the case where we instead want to incorporate information from **two** vectors v_a and v_b, with corresponding key vectors k_a and k_b. Assume that (1) all key vectors are orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1. **Find an expression** for a query vector q such that $c \approx \frac{1}{2}(v_a + v_b)$, and **justify your answer**.* (Recall what you learned in part (a).)

Answer: Choose $q = \lambda(k_a + k_b)$ for a large scalar $\lambda > 0$. Because the keys are orthonormal, $k_i^\top q = \lambda\left(k_i^\top k_a + k_i^\top k_b\right)$ equals λ for $i \in \{a, b\}$ and 0 for all other i. The attention weights are therefore

$$\alpha_a = \alpha_b = \frac{e^\lambda}{2e^\lambda + (n-2)} \xrightarrow{\lambda \to \infty} \frac{1}{2}, \qquad \alpha_i \xrightarrow{\lambda \to \infty} 0 \quad (i \notin \{a, b\}),$$

so $c = \sum_{i=1}^{n} \alpha_i v_i \approx \frac{1}{2}(v_a + v_b)$, and the approximation becomes exact as $\lambda \to \infty$.

(c) (5 points) **Drawbacks of single-headed attention:** In the previous part, we saw how it was *possible* for a single-headed attention to focus equally on two values. The same concept could easily be extended to any subset of values. In this question we'll see why it's not a *practical* solution.

Consider a set of key vectors $\{k_1, \ldots, k_n\}$ that are now randomly sampled, $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, where the means $\mu_i \in \mathbb{R}^d$ are known to you, but the covariances Σ_i are unknown (unless specified otherwise

*Hint: while the softmax function will never *exactly* average the two vectors, you can get close by using a large scalar multiple in the expression.

in the question). Further, assume that the means μ_i are all perpendicular; $\mu_i^\top \mu_j = 0$ if $i \neq j$, and unit norm, $\|\mu_i\| = 1$.

i. (2 points) Assume that the covariance matrices are $\Sigma_i = \alpha I, \forall i \in \{1, 2, \ldots, n\}$, for vanishingly small α. Design a query q in terms of the μ_i such that as before, $c \approx \frac{1}{2}(v_a + v_b)$, and provide a brief argument as to why it works.

Answer: Take $q = \lambda(\mu_a + \mu_b)$ for a large $\lambda > 0$. With $\Sigma_i = \alpha I$ for vanishingly small α, every sampled key is a tiny perturbation of its mean, $k_i \approx \mu_i$. Hence, by the orthogonality and unit norm of the μ_i, we get $k_a^\top q \approx \lambda$ and $k_b^\top q \approx \lambda$ while $k_i^\top q \approx 0$ for $i \notin \{a, b\}$, with errors of size $O(\sqrt{\alpha})$. Exactly as in part (b), the softmax then concentrates almost all of its mass, equally, on a and b, giving $c \approx \frac{1}{2}(v_a + v_b)$.

ii. (3 points) Though single-headed attention is resistant to small perturbations in the keys, some types of larger perturbations may pose a bigger issue. In some cases, one key vector k_a may be larger or smaller in norm than the others, while still pointing in the same direction as μ_a.[†]
As an example, let us consider a covariance for item a as $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ for vanishingly small α (as shown in figure 1). This causes k_a to point in roughly the same direction as μ_a, but with large variances in magnitude. Further, let $\Sigma_i = \alpha I$ for all $i \neq a$.

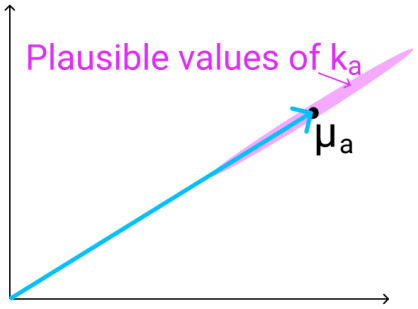

Figure 1: The vector μ_a (shown here in 2D as an example), with the range of possible values of k_a shown in red. As mentioned previously, k_a points in roughly the same direction as μ_a, but may have larger or smaller magnitude.

When you sample $\{k_1, \ldots, k_n\}$ multiple times, and use the q vector that you defined in part i., what do you expect the vector c will look like qualitatively for different samples? Think about how it differs from part (i) and how c's variance would be affected.

Answer: Now k_a has a large variance along its own mean direction: $\mathrm{Var}\left(k_a^\top \mu_a\right) = \mu_a^\top \Sigma_a \mu_a = \alpha + \frac{1}{2} \approx \frac{1}{2}$, so $k_a^\top \mu_a$ fluctuates around 1 with standard deviation $\approx 1/\sqrt{2}$, whereas $k_b^\top \mu_b \approx 1$ is stable. The two scores that compete in the softmax are $k_a^\top q = \lambda\left(k_a^\top \mu_a\right)$ and $k_b^\top q \approx \lambda$, so their difference $\lambda\left(k_a^\top \mu_a - 1\right)$ is random with standard deviation $\approx \lambda/\sqrt{2}$, which is huge because λ is large. Since softmax amplifies score differences exponentially, whenever $k_a^\top \mu_a > 1$ we get $c \approx v_a$, whenever $k_a^\top \mu_a < 1$ we get $c \approx v_b$, and near-equal scores (which would give $\approx \frac{1}{2}(v_a + v_b)$) are rare. So, across samples, c no longer stably equals $\frac{1}{2}(v_a + v_b)$ as in part (i); instead it has very large variance, flipping between $\approx v_a$ and $\approx v_b$.

(d) (3 points) **Benefits of multi-headed attention:** Now we'll see some of the power of multi-headed attention. We'll consider a simple version of multi-headed attention which is identical to single-headed self-attention as we've presented it, except two query vectors (q_1 and q_2) are defined, which leads to a pair of vectors (c_1 and c_2), each the output of single-headed attention given its respective query vector. The final output of the multi-headed attention is their average, $\frac{1}{2}(c_1 + c_2)$.

[†]Unlike the original Transformer, newer Transformer models apply layer normalization before attention. In these pre-layernorm models, norms of keys cannot be too different which makes the situation in this question less likely to occur.

As in question 1(c), consider a set of key vectors $\{k_1, \ldots, k_n\}$ that are randomly sampled, $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, where the means μ_i are known to you, but the covariances Σ_i are unknown. Also as before, assume that the means μ_i are mutually orthogonal; $\mu_i^\top \mu_j = 0$ if $i \neq j$, and unit norm, $\|\mu_i\| = 1$.

i. (1 point) Assume that the covariance matrices are $\Sigma_i = \alpha I$, for vanishingly small α. Design q_1 and q_2 in terms of μ_i such that c is approximately equal to $\frac{1}{2}(v_a + v_b)$. Note that q_1 and q_2 should have different expressions.

Answer: Choose $q_1 = \lambda\mu_a$ and $q_2 = \lambda\mu_b$ with large $\lambda > 0$ (so $q_1 \neq q_2$). For head 1, only key a has a large score ($k_a^\top q_1 \approx \lambda$; all others ≈ 0), so $c_1 \approx v_a$. For head 2, only key b has a large score ($k_b^\top q_2 \approx \lambda$), so $c_2 \approx v_b$. Their average is therefore $c = \frac{1}{2}(c_1 + c_2) \approx \frac{1}{2}(v_a + v_b)$.

ii. (2 points) Assume that the covariance matrices are $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ for vanishingly small α, and $\Sigma_i = \alpha I$ for all $i \neq a$. Take the query vectors q_1 and q_2 that you designed in part i. What, qualitatively, do you expect the output c to look like across different samples of the key vectors? Explain briefly in terms of variance in c_1 and c_2. You can ignore cases in which $k_a^\top q_i < 0$.

Answer: Head 1 uses $q_1 = \lambda\mu_a$, so its only nonzero score is $k_a^\top q_1 = \lambda\,(k_a^\top \mu_a)$, which remains positive (we ignore the negative cases). No matter how the magnitude of k_a fluctuates, all other scores are ≈ 0, so the softmax still concentrates on a: $c_1 \approx v_a$ for every sample, with small variance — the fluctuation only changes how sharply peaked α_1 is, not *which* value is copied. Head 2 uses $q_2 = \lambda\mu_b$; key b is unperturbed ($\Sigma_b = \alpha I$), so $c_2 \approx v_b$ with small variance as well. Since the two heads produce independent, stable outputs, the average $c = \frac{1}{2}(c_1 + c_2) \approx \frac{1}{2}(v_a + v_b)$ is stable across samples: the variance of c is small (it is $\frac{1}{4}\,(\mathrm{Var}(c_1) + \mathrm{Var}(c_2))$), unlike the single-head case in part (c)(ii).

(e) (1 point) Based on part (d), briefly summarize how multi-headed attention overcomes the drawbacks of single-headed attention that you identified in part (c).

Answer: The drawback of single-headed attention is that one query must be aligned with *several* keys at once, so a perturbation of any attended key's norm is exponentially amplified by the softmax and the output becomes unstable (part (c)(ii)). Multi-headed attention decomposes the desired combination into separate heads, each with its own query aligned to a *single* key direction: each head's attention then concentrates on one value regardless of how that key's norm fluctuates (low variance per head), and averaging the stable heads produces the desired mixture $\frac{1}{2}(v_a + v_b)$ with greatly reduced variance.

2. Position Embeddings Exploration (6 points)

Position embeddings are an important component of the Transformer architecture, allowing the model to differentiate between tokens based on their position in the sequence. In this question, we'll explore the need for positional embeddings in Transformers and how they can be designed.

Recall that the crucial components of the Transformer architecture are the self-attention layer and the feed-forward neural network layer. Given an input tensor $\mathbf{X} \in \mathbb{R}^{T \times d}$, where T is the sequence length and d is the hidden dimension, the self-attention layer computes the following:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$
$$\mathbf{H} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$ are weight matrices, and $\mathbf{H} \in \mathbb{R}^{T \times d}$ is the output.

Next, the feed-forward layer applies the following transformation:

$$\mathbf{Z} = \mathrm{ReLU}(\mathbf{H}\mathbf{W}_1 + \mathbf{1} \cdot \mathbf{b}_1)\mathbf{W}_2 + \mathbf{1} \cdot \mathbf{b}_2$$

where $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^{1\times d}$ are weights and biases; $\mathbf{1} \in \mathbb{R}^{T\times 1}$ is a vector of ones[‡]; and $\mathbf{Z} \in \mathbb{R}^{T\times d}$ is the final output.

(Note that we have omitted some details of the Transformer architecture for simplicity.)

(a) (4 points) **Permuting the input.**

i. (3 points) Suppose we permute the input sequence $\mathbf{X}$ such that the tokens are shuffled randomly. This can be represented as multiplication by a permutation matrix $\mathbf{P} \in \mathbb{R}^{T\times T}$, i.e. $\mathbf{X}_{\text{perm}} = \mathbf{PX}$. (See Wikipedia for a recap on permutation matrices.)
Show that the output $\mathbf{Z}_{\text{perm}}$ for the permuted input $\mathbf{X}_{\text{perm}}$ will be $\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$.
You are given that for any permutation matrix $\mathbf{P}$ and any matrix $\mathbf{A}$, the following hold: $\text{softmax}(\mathbf{PAP}^\top) = \mathbf{P}\,\text{softmax}(\mathbf{A})\,\mathbf{P}^\top$ and $\text{ReLU}(\mathbf{PA}) = \mathbf{P}\,\text{ReLU}(\mathbf{A})$.
Answer: Let $\mathbf{Q} = \mathbf{XW}_Q$, $\mathbf{K} = \mathbf{XW}_K$, $\mathbf{V} = \mathbf{XW}_V$ (so that $\mathbf{H} = \text{softmax}(\mathbf{QK}^\top/\sqrt{d})\,\mathbf{V}$). Since $\mathbf{X}_{\text{perm}} = \mathbf{PX}$, we have $\mathbf{Q}_{\text{perm}} = \mathbf{PQ}$, $\mathbf{K}_{\text{perm}} = \mathbf{PK}$, and $\mathbf{V}_{\text{perm}} = \mathbf{PV}$. Applying the given softmax identity with $\mathbf{A} = \mathbf{QK}^\top/\sqrt{d}$ and using $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$,

$$\mathbf{H}_{\text{perm}} = \text{softmax}\left(\frac{\mathbf{PQK}^\top\mathbf{P}^\top}{\sqrt{d}}\right)\mathbf{PV} = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right)\mathbf{P}^\top\mathbf{PV} = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right)\mathbf{V} = \mathbf{PH}.$$

For the feed-forward layer, note that $\mathbf{P1} = \mathbf{1}$ (a permutation matrix maps the all-ones vector to itself), so $\mathbf{1b}_1 = \mathbf{P1b}_1$ and $\mathbf{1b}_2 = \mathbf{P1b}_2$. Hence, applying the given ReLU identity with $\mathbf{A} = \mathbf{HW}_1 + \mathbf{1b}_1$,

$$\begin{aligned}\mathbf{Z}_{\text{perm}} &= \text{ReLU}(\mathbf{PHW}_1 + \mathbf{1b}_1)\,\mathbf{W}_2 + \mathbf{1b}_2 = \text{ReLU}(\mathbf{P}\,(\mathbf{HW}_1 + \mathbf{1b}_1))\,\mathbf{W}_2 + \mathbf{P1b}_2 \\ &= \mathbf{P}\,\text{ReLU}(\mathbf{HW}_1 + \mathbf{1b}_1)\,\mathbf{W}_2 + \mathbf{P1b}_2 = \mathbf{P}\,[\text{ReLU}(\mathbf{HW}_1 + \mathbf{1b}_1)\,\mathbf{W}_2 + \mathbf{1b}_2] = \mathbf{PZ}.\end{aligned}$$

So the (position-embedding-free) Transformer is permutation-equivariant: shuffling the input tokens merely shuffles the output rows.

ii. (1 point) Think about the implications of the result you derived in part i. **Explain** why this property of the Transformer model could be problematic when processing text.
Answer: Because the model is permutation-equivariant, it cannot use word order at all: it would produce the same (merely reordered) output whether the input is a sentence or any shuffling of its words, so it treats "dog bites man" and "man bites dog" identically. In natural language, however, word order is crucial — it determines argument roles and meaning — so a model that ignores order cannot correctly model the semantics of text.

(b) (2 points) **Position embeddings** are vectors that encode the position of each token in the sequence. They are added to the input word embeddings before feeding them into the Transformer.

One approach is to generate position embedding using a fixed function of the position and the dimension of the embedding. If the input word embeddings are $\mathbf{X} \in \mathbb{R}^{T\times d}$, the position embeddings $\Phi \in \mathbb{R}^{T\times d}$ are generated as follows:

$$\Phi_{(t,2i)} = \sin\left(t/10000^{2i/d}\right)$$
$$\Phi_{(t,2i+1)} = \cos\left(t/10000^{2i/d}\right)$$

where $t \in \{0, 1, \ldots T-1\}$ and $i \in \{0, 1, \ldots d/2-1\}$[§].

Specifically, the position embeddings are added to the input word embeddings:

$$\mathbf{X}_{\text{pos}} = \mathbf{X} + \Phi$$

[‡]Outer product with $\mathbf{1}$ represents broadcasting operation and makes feed forward network notations mathematically sound.
[§]Here d is assumed even which is typically the case for most models.

i. (1 point) Do you think the position embeddings will help the issue you identified in part (a)? If yes, explain how and if not, explain why not.
Answer: Yes. After adding the position embeddings, the input becomes $\mathbf{X}_{\text{pos}} = \mathbf{X}+\mathbf{\Phi}$, where $\mathbf{\Phi}$ encodes the absolute position of each token and is *not* permuted along with the tokens. Hence permuting the tokens now gives $\mathbf{PX} + \mathbf{\Phi} \neq \mathbf{P}(\mathbf{X} + \mathbf{\Phi})$ in general, so the output is no longer simply the permuted output of the original input: the position-dependent signal breaks the permutation equivariance, and the model can now distinguish different orderings of the same tokens.

ii. (1 point) Can the position embeddings for two different tokens in the input sequence be the same? If yes, provide an example. If not, explain why not.
Answer: Yes. The embeddings are built from sine and cosine, which are periodic functions of the position t, so two different positions can receive the same embedding. For example, in the simplest case $d = 2$, $\mathbf{\Phi}(t) = (\sin t, \cos t)$ is periodic with period 2π, so positions t and $t + 2\pi k$ ($k \in \mathbb{Z}$) yield identical embeddings. More generally, the i-th frequency pair $\left(\sin\left(t/10000^{2i/d}\right), \cos\left(t/10000^{2i/d}\right)\right)$ repeats every $2\pi \cdot 10000^{2i/d}$ positions, so if the sequence is long enough for two positions to differ by a common period of all the sinusoids, their full position embeddings coincide (and even before an exact collision, far-apart positions can receive very similar — and, with floating-point precision, eventually numerically equal — embeddings).

3. Pretrained Transformer models and knowledge access (35 points)

You'll train a Transformer to perform a task that involves accessing knowledge about the world — knowledge which isn't provided via the task's training data (at least if you want to generalize outside the training set). You'll find that it more or less fails entirely at the task. You'll then learn how to pretrain that Transformer on Wikipedia text that contains world knowledge, and find that finetuning that Transformer on the same knowledge-intensive task enables the model to access some of the knowledge learned at pretraining time. You'll find that this enables models to perform considerably above chance on a held out development set.

The code you're provided with is a fork of Andrej Karpathy's `minGPT`. It's nicer than most research code in that it's relatively simple and transparent. The "GPT" in `minGPT` refers to the Transformer language model of OpenAI, originally described in this paper [1].

As in previous assignments, you will want to develop on your machine locally, then run training on GCP/Colab. You can use the same conda environment from previous assignments for local development, and the same process for training on a GPU.[¶]

You'll need around 3 hours for training, so budget your time accordingly! We have provided a sample Colab with the the commands that require GPU training. **Note that dataset multi-processing can fail on local machines without GPU, so to debug locally, you might have to change `num_workers` to 0.**

Your work with this codebase is as follows:

(a) (0 points) **Check out the demo.**
In the `mingpt-demo/` folder is a Jupyter notebook `play_char.ipynb` that trains and samples from a Transformer language model. Take a look at it (locally on your computer) to get somewhat familiar with how it defines and trains models. Some of the code you're writing below will be inspired by what you see in this notebook.

Note that you do not have to write any code, run the notebook or submit written answers for this part.

[¶]See CS224n GCP Guide for a refresher on GCP.

(b) (0 points) **Read through `NameDataset` in `src/dataset.py`, our dataset for reading name-birthplace pairs.**

The task we'll be working on with our pretrained models is attempting to access the birth place of a notable person, as written in their Wikipedia page. We'll think of this as a particularly simple form of question answering:

Q: Where was [person] born?
A: [place]

From now on, you'll be working with the `src/` folder. The code in `mingpt-demo/` won't be changed or evaluated for this assignment. In `dataset.py`, you'll find the the class `NameDataset`, which reads a TSV (tab-separated values) file of name/place pairs and produces examples of the above form that we can feed to our Transformer model.

To get a sense of the examples we'll be working with, if you run the following code, it'll load your `NameDataset` on the training set `birth_places_train.tsv` and print out a few examples.

```
python src/dataset.py namedata
```

Note that you do not have to write any code or submit written answers for this part.

(c) (0 points) **Implement finetuning (without pretraining).**

Take a look at `run.py`. It has some skeleton code specifying flags you'll eventually need to handle as command line arguments. In particular, you might want to *pretrain*, *finetune*, or *evaluate* a model with this code. For now, we'll focus on the finetuning function, in the case without pretraining.

Taking inspiration from the training code in the `play_char.ipynb` file, write code to finetune a Transformer model on the name/birthplace dataset, via examples from the `NameDataset` class. For now, implement the case without pretraining (i.e. create a model from scratch and train it on the birthplace prediction task from part (b)). You'll have to modify two sections, marked `[part c]` in the code: one to initialize the model, and one to finetune it. Note that you only need to initialize the model in the case labeled "vanilla" for now (later in section (g), we will explore a model variant). Use the hyperparameters for the `Trainer` specified in the `run.py` code.

Also take a look at the *evaluation* code which has been implemented for you. It samples predictions from the trained model and calls `evaluate_places()` to get the total percentage of correct place predictions. You will run this code in part (d) to evaluate your trained models.

This is an intermediate step for later portions, including Part d, which contains commands you can run to check your implementation. No written answer is required for this part.

Hint: Both `run.py` and `play_char.ipynb` use minGPT so the code for this part will be similar to the training code in `play_char.ipynb`.

(d) (4 points) **Make predictions (without pretraining).**

Train your model on `birth_places_train.tsv`, and evaluate on `birth_dev.tsv`. Specifically, you should now be able to run the following three commands:

```
# Train on the names dataset
python src/run.py finetune vanilla wiki.txt \
       --writing_params_path vanilla.model.params \
       --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set, writing out predictions
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.model.params \
       --eval_corpus_path birth_dev.tsv \
       --outputs_path vanilla.nopretrain.dev.predictions
```

```
# Evaluate on the test set, writing out predictions
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.model.params \
       --eval_corpus_path birth_test_inputs.tsv \
       --outputs_path vanilla.nopretrain.test.predictions
```

Training will take less than 10 minutes (on GCP). Report your model's accuracy on the dev set (as printed by the second command above). Similar to assignment 3, we also have Tensorboard logging in assignment 4 for debugging. It can be launched using `tensorboard --logdir expt/`. Don't be surprised if it is well below 10%; we will be digging into why in Part 4. As a reference point, we want to also calculate the accuracy the model would have achieved if it had just predicted "London" as the birth place for everyone in the dev set. Fill in `london_baseline.py` to calculate the accuracy of that approach and report your result in the file. You should be able to leverage existing code such that the file is only a few lines long.

(e) (10 points) **Define a *span corruption* function for pretraining.**
In the file `src/dataset.py`, implement the `__getitem__()` function for the dataset class `CharCorruptionDataset`. Follow the instructions provided in the comments in `dataset.py`. Span corruption is explored in the T5 paper [2]. It randomly selects spans of text in a document and replaces them with unique tokens (noising). Models take this noised text, and are required to output a pattern of each unique sentinel followed by the tokens that were replaced by that sentinel in the input. In this question, you'll implement a simplification that only masks out a single sequence of characters.

This question will be graded via autograder based on whether your span corruption function implements some basic properties of our spec. We'll instantiate the `CharCorruptionDataset` with our own data, and draw examples from it.

To help you debug, if you run the following code, it'll sample a few examples from your `CharCorruptionDataset` on the pretraining dataset `wiki.txt` and print them out for you.

```
python src/dataset.py charcorruption
```

(f) (10 points) **Pretrain, finetune, and make predictions. Budget about 1 hour for training.**
Now fill in the *pretrain* portion of `run.py`, which will pretrain a model on the span corruption task. Additionally, modify your *finetune* portion to handle finetuning in the case *with* pretraining. In particular, if a path to a pretrained model is provided in the bash command, load this model before finetuning it on the birthplace prediction task. Pretrain your model on `wiki.txt` (which should take approximately 40-60 minutes), finetune it on `NameDataset` and evaluate it. Specifically, you should be able to run the following four commands:

```
# Pretrain the model
python src/run.py pretrain vanilla wiki.txt \
       --writing_params_path vanilla.pretrain.params

# Finetune the model
python src/run.py finetune vanilla wiki.txt \
       --reading_params_path vanilla.pretrain.params \
       --writing_params_path vanilla.finetune.params \
       --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate vanilla wiki.txt \
```

```
       --reading_params_path vanilla.finetune.params \
       --eval_corpus_path birth_dev.tsv \
       --outputs_path vanilla.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.finetune.params \
       --eval_corpus_path birth_test_inputs.tsv \
       --outputs_path vanilla.pretrain.test.predictions
```

We expect the dev accuracy will be at least 15%, and will expect a similar accuracy on the held out test set.

(g) (11 points) **Write and try out a different kind of position embeddings (Budget about 1 hour for training)**

In the previous part, you used the vanilla Transformer model, which used learned positional embeddings. In the written part, you also learned about the sinusoidal positional embeddings used in the original Transformer paper. In this part, you'll implement a different kind of positional embedding, called *RoPE* (Rotary Positional Embedding) [3].

RoPE is a fixed positional embedding that is designed to encode relative position rather than absolute position. The issue with absolute positions is that if the transformer won't perform well on context lengths (e.g. 1000) much larger than it was trained on (e.g. 128), because the distribution of the position embeddings will be very different from the ones it was trained on. Relative position embeddings like RoPE alleviate this issue.

Given a feature vector with two features $x_t^{(1)}$ and $x_t^{(2)}$ at position t in the sequence, the RoPE positional embedding is defined as:

$$\text{RoPE}(x_t^{(1)}, x_t^{(2)}, t) = \begin{pmatrix} \cos t\theta & -\sin t\theta \\ \sin t\theta & \cos t\theta \end{pmatrix} \begin{pmatrix} x_t^{(1)} \\ x_t^{(2)} \end{pmatrix}$$

where θ is a fixed angle. For two features, the RoPE operation corresponds to a 2D rotation of the features by an angle $t\theta$. Note that the angle is a function of the position t.

For a d dimensional feature, RoPE is applied to each pair of features with an angle θ_i defined as $\theta_i = 10000^{-2(i-1)/d},\ i \in \{1, 2, \ldots, d/2\}$.

$$\begin{pmatrix} \cos t\theta_1 & -\sin t\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin t\theta_1 & \cos t\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos t\theta_2 & -\sin t\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin t\theta_2 & \cos t\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos t\theta_{d/2} & -\sin t\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin t\theta_{d/2} & \cos t\theta_{d/2} \end{pmatrix} \begin{pmatrix} x_t^{(1)} \\ x_t^{(2)} \\ x_t^{(3)} \\ x_t^{(4)} \\ \vdots \\ x_t^{(d-1)} \\ x_t^{(d)} \end{pmatrix} \tag{3}$$

Finally, instead of adding the positional embeddings to the input embeddings, RoPE is applied to the key and query vectors for each head in the attention block for all the Transformer layers.

i. (2 points) Using the rotation interpretation, RoPE operation can be viewed as rotation of the complex number $x_t^{(1)} + ix_t^{(2)}$ by an angle $t\theta$. Recall that this corresponds to multiplication by $e^{it\theta} = \cos t\theta + i \sin t\theta$.

For higher dimensional feature vectors, this interpretation allows us to compute Equation 3 more efficiently. Specifically, we can rewrite the RoPE operation as an element-wise multiplication (denoted by $\odot$) of two vectors as follows:

$$\begin{pmatrix} \cos t\theta_1 + i \sin t\theta_1 \\ \cos t\theta_2 + i \sin t\theta_2 \\ \vdots \\ \cos t\theta_{d/2} + i \sin t\theta_{d/2} \end{pmatrix} \odot \begin{pmatrix} x_t^{(1)} + i x_t^{(2)} \\ x_t^{(3)} + i x_t^{(4)} \\ \vdots \\ x_t^{(d-1)} + i x_t^{(d)} \end{pmatrix} \tag{4}$$

Show that the elements of the vector in Equation 3 can be obtained from Equation 4. Note that some additional operations like reshaping are necessary to make the two expressions equal but you do not need to provide a detailed derivation for full points.

Answer: For each consecutive pair of features $\left(x_t^{(2i-1)}, x_t^{(2i)}\right)$, $i \in \{1, \ldots, d/2\}$, Equation 3 applies the 2×2 rotation

$$\begin{pmatrix} \cos t\theta_i & -\sin t\theta_i \\ \sin t\theta_i & \cos t\theta_i \end{pmatrix} \begin{pmatrix} x_t^{(2i-1)} \\ x_t^{(2i)} \end{pmatrix} = \begin{pmatrix} x_t^{(2i-1)} \cos t\theta_i - x_t^{(2i)} \sin t\theta_i \\ x_t^{(2i-1)} \sin t\theta_i + x_t^{(2i)} \cos t\theta_i \end{pmatrix}.$$

Writing the pair as the complex number $z_i = x_t^{(2i-1)} + i\, x_t^{(2i)}$, Equation 4 computes

$$(\cos t\theta_i + i \sin t\theta_i)\left(x_t^{(2i-1)} + i x_t^{(2i)}\right) = \underbrace{x_t^{(2i-1)} \cos t\theta_i - x_t^{(2i)} \sin t\theta_i}_{\text{real part}} + i \underbrace{\left(x_t^{(2i-1)} \sin t\theta_i + x_t^{(2i)} \cos t\theta_i\right)}_{\text{imaginary part}},$$

so the real part equals the first rotated component and the imaginary part equals the second. Hence the two expressions give the same elements: reshape the d-dimensional vector into $d/2$ complex numbers (pairing features $(2i-1, 2i)$), multiply element-wise by $e^{it\theta_i} = \cos t\theta_i + i \sin t\theta_i$, then view the resulting complex numbers back as real pairs and flatten.

ii. (1 point) **Relative Embeddings.** Now we will show that the dot product of the RoPE embeddings of two vectors at positions t_1 and t_2 depends on the relative position $t_1 - t_2$ only.
For simiplicity, we will assume two dimensional feature vectors (eg. $[a, b]$) and work with their complex number representations (eg. $a + ib$).
Show that $\langle \text{RoPE}(z_1, t_1), \text{RoPE}(z_2, t_2) \rangle = \langle \text{RoPE}(z_1, t_1 - t_2), \text{RoPE}(z_2, 0) \rangle$ where $\langle \cdot, \cdot \rangle$ denotes the dot product and $\text{RoPE}(z, t)$ is the RoPE embedding of vector z at position t.
(Hint: Dot product of vectors represented as complex numbers is given by $\langle z_1, z_2 \rangle = \text{Re}(\overline{z_1} z_2)$. For a complex number $z = a + ib$ $(a, b \in \mathbb{R})$, $\text{Re}(z) = a$ indicates the real component of z and $\bar{z} = a - ib$ is the complex conjugate of z.)

Answer: RoPE rotates the complex representation by the position-dependent angle $t\theta$, i.e. $\text{RoPE}(z, t) = z\, e^{it\theta}$. Using the hint $\langle z_1, z_2 \rangle = \text{Re}(\overline{z_1} z_2)$,

$$\langle \text{RoPE}(z_1, t_1), \text{RoPE}(z_2, t_2) \rangle = \text{Re}\left(\overline{z_1 e^{it_1\theta}} \cdot z_2 e^{it_2\theta}\right) = \text{Re}\left(\overline{z_1} z_2\, e^{i(t_2 - t_1)\theta}\right),$$

while

$$\langle \text{RoPE}(z_1, t_1 - t_2), \text{RoPE}(z_2, 0) \rangle = \text{Re}\left(\overline{z_1 e^{i(t_1 - t_2)\theta}} \cdot z_2\right) = \text{Re}\left(\overline{z_1} z_2\, e^{-i(t_1 - t_2)\theta}\right) = \text{Re}\left(\overline{z_1} z_2\, e^{i(t_2 - t_1)\theta}\right).$$

The two expressions are identical: they both equal $\text{Re}\left(\overline{z_1} z_2\, e^{i(t_2 - t_1)\theta}\right)$, which depends only on the relative position $t_1 - t_2$ (through the phase $e^{i(t_2 - t_1)\theta}$), not on the absolute positions.

iii. (8 points) In the provided code, RoPE is implemented using the functions `precompute_rotary_emb` and `apply_rotary_emb` in `src/attention.py`. You need to implement these functions and the parts of code marked `[part g]` in `src/attention.py` and `src/run.py` to use RoPE in the model.

Train a model with RoPE on the span corruption task and finetune it on the birthplace prediction task. Specifically, you should be able to run the following four commands:

```
# Pretrain the model
python src/run.py pretrain rope wiki.txt \
       --writing_params_path rope.pretrain.params

# Finetune the model
python src/run.py finetune rope wiki.txt \
       --reading_params_path rope.pretrain.params \
       --writing_params_path rope.finetune.params \
       --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate rope wiki.txt \
       --reading_params_path rope.finetune.params \
       --eval_corpus_path birth_dev.tsv \
       --outputs_path rope.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate rope wiki.txt \
       --reading_params_path rope.finetune.params \
       --eval_corpus_path birth_test_inputs.tsv \
       --outputs_path rope.pretrain.test.predictions
```

We'll score your model as to whether it gets at least 30% accuracy on the test set, which has answers held out.

4. Considerations in pretrained knowledge (5 points)

Please type the answers to these written questions (to make TA lives easier).

(a) (1 point) Succinctly explain why the pretrained (vanilla) model was able to achieve an accuracy of above 10%, whereas the non-pretrained model was not.

Answer: The non-pretrained model is trained from scratch only on the finetuning corpus (a few thousand name–birthplace pairs). Since the input "Where was X born?" contains no information about the answer, the model cannot generalize beyond memorizing training examples, so on unseen names it collapses toward the prior over common birthplaces (roughly the London baseline of $\sim$5%). The pretrained model, by contrast, first learns span corruption on a large Wikipedia corpus, where it acquires statistical knowledge about people and places, including name–birthplace associations. Finetuning then teaches it to express that stored knowledge in the question–answer format, allowing it to retrieve plausible birthplaces for names it has knowledge about, which pushes dev accuracy well above chance ($\geq$ 15%).

(b) (2 points) Take a look at some of the correct predictions of the pretrain+finetuned vanilla model, as well as some of the errors. We think you'll find that it's impossible to tell, just looking at the output, whether the model *retrieved* the correct birth place, or *made up* an incorrect birth place. Consider the implications of this for user-facing systems that involve pretrained NLP components. Come up with two **distinct** reasons why this model behavior (i.e. unable to tell whether it's retrieved or made up) may cause concern for such applications, and an example for each reason.

Answer: **(1) Misinformation to users.** Correct retrievals and hallucinated answers are produced in the same fluent, confident form, so users (and any downstream system) cannot distinguish

fact from fabrication and will treat invented answers with the same authority as true ones. *Example:* a user-facing assistant answers "Where was person X born?" with a confidently stated but wrong city; the user, having no way to tell it was invented, may propagate the false fact (e.g., in a biography or travel plan).

(2) Lack of auditability and accountability. Because one cannot tell whether an output was recalled from the model's (pretraining) knowledge or invented at generation time, errors cannot be attributed to the data or to the model, which makes debugging, oversight, and accountability difficult; in addition, "retrieved" answers can silently carry biases, outdated facts, or sensitive information from the pretraining corpus, and "made-up" answers can carry stereotyped associations, all looking identical in the output. *Example:* a downstream fact-checking or screening system ingests these predictions and cannot flag which claims require verification, so biased or false claims flow onward unquestioned, and systematic errors go unnoticed.

(c) (2 points) If your model didn't see a person's name at pretraining time, and that person was not seen at fine-tuning time either, it is not possible for it to have "learned" where they lived. Yet, your model will produce *something* as a predicted birth place for that person's name if asked. Concisely describe a strategy your model might take for predicting a birth place for that person's name, and one ethical concern this raises for the use of such applications.
(While 4b discussed the problems that could arise from made up predictions, 4c asks for a mechanism the model could be using for generating birth places of people not seen at fine-tuning time and why such a mechanism could be problematic.)

Answer: **Strategy.** Having never seen the person, the model cannot retrieve a stored fact. Instead, it conditions on the question template and on surface features of the name (spelling and phonetic patterns, token overlaps with names seen during pretraining/finetuning) and generates the birth place that maximizes likelihood under the name→place distribution it learned — effectively an analogical guess: names that resemble those of a given country/ethnicity are mapped to that country's common cities, or, failing any strong cue, the model falls back to the most frequent places in its training distribution.

Ethical concern. This mechanism makes judgments about people from superficial features of their names, which is a form of stereotyping: it can misattribute nationality or ethnicity, perpetuate demographic stereotypes, and, when deployed in applications such as automated background checks, hiring screens, or profile/search systems, can produce discriminatory or defamatory outputs about real individuals based purely on how their name looks.

Submission Instructions

You will submit this assignment on GradeScope as two submissions – one for **Assignment 4 [coding]** and another for **Assignment 4 [written]**:

1. Verify that the following files exist at these specified paths within your assignment directory:
   - The no-pretraining model and predictions: `vanilla.model.params`, `vanilla.nopretrain.dev.predictions`, `vanilla.nopretrain.test.predictions`
   - The London baseline accuracy: `london_baseline_accuracy.txt`
   - The pretrain-finetune model and predictions: `vanilla.finetune.params`, `vanilla.pretrain.dev.predictions`, `vanilla.pretrain.test.predictions`
   - The RoPE model and predictions: `rope.finetune.params`, `rope.pretrain.dev.predictions`, `rope.pretrain.test.predictions`
2. Run `collect_submission.sh` (on Linux/Mac) or `collect_submission.bat` (on Windows) to produce your `assignment4.zip` file.

3. Upload your `assignment4.zip` file to GradeScope to **Assignment 4 [coding]**.
4. Check that the public autograder tests passed correctly.
5. Upload your written solutions, for questions 1, parts of 2, and 3, to GradeScope to **Assignment 4 [written]**. Tag it properly!

References

[1] Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding with unsupervised learning. *Technical report, OpenAI* (2018).

[2] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research 21*, 140 (2020), 1–67.

[3] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing 568* (2024), 127063.